🚀 React Day 9

UseFormStatus Hook

→ What is useFormStatus?

useFormStatus() is a React hook that gives you access to the **current status of a form submission**, like whether it's **pending** (in progress), **succeeded**, or **failed**.

It is only usable inside a Form component's subtree, **especially with <form action={someServerAction}>**.

```
import React from 'react'
import {useFormStatus} from 'react-dom'

const UseFromStatus = () => {

    const handleSubmit=async ()=>{
        await new Promise(res=>setTimeout(res,2000));
        console.log("submit")
    }
    function CustomerForm(){
        const {pending} = useFormStatus();
        console.log(pending);

        return(
            <div>
                <input type="text" />
                <br />
                <input type="text" />
                <br />
                <button disabled={pending}>{pending?"submitting...":"submit"}</button>
            </div>
        )
    }

  return (
    <div>

        <form action={handleSubmit}>
            <CustomerForm />
        </form>

    </div>
  )
}

export default UseFromStatus
```

UseTransition Hook

```
import React from 'react'
import { useState } from 'react'

const UseTransition = () => {

    const[pending,setPending] = useState(false);
    const handlePending=async ()=>{
        setPending(true);
        await new Promise(res=>setTimeout(res,2000))
        setPending(false)
    }
  return (
    <div>
        <h3>Use Transition</h3>
        <button onClick={handlePending} disabled={pending}>Click</button>
    </div>
  )
}

export default UseTransition
```

this is without usetransition, in this we have to play with the useTransition

```
import React, { useTransition } from 'react';

const UseFormStatus = () => {
  const [isPending, startTransition] = useTransition();

  const handleSubmit = (e) => {
    e.preventDefault();

    startTransition(() => {
      setTimeout(() => {
        alert('Form submitted!');
      }, 2000);
    });
  };

  return (
    <div style={{ textAlign: 'center', marginTop: '50px' }}>
      <form onSubmit={handleSubmit}>
        <input type="text" placeholder="Name" required />
        <br /><br />
        <input type="text" placeholder="Email" required />
        <br /><br />
        <button type="submit" disabled={isPending} style={{ padding: '10px 20px' }}>
          {isPending ? <div className="spinner"></div> : 'Submit'}
        </button>
      </form>
    </div>
  );
};

export default UseFormStatus;
```

with useTransition

Pure Function in React

- Ye ek **simple function** hota hai.
- **Same input doge to same output hi dega**.
- Ye function **bahar ki duniya mein kuch change nahi karta** – jaise console.log, API call, ya koi variable change.

Example:

const add = (a, b) => a + b;

Agar har baar add(2, 3) likhoge to hamesha 5 hi milega. Ye hai **pure function**.

Pure Components

- Ye ek **React component** hota hai.
- Agar usko **same props milte hain**, to wo **dobara render nahi hota**.
- Ye React ke performance ko **fast** banata hai.

Example:

```
const Welcome = React.memo(({ name }) => {

  return <h1>Hello {name}</h1>;

});
```

Agar name same hai, to ye component **dobara nahi chalega**. Ye hai **pure component**.

🎯 Difference in One Line:

Pure Function	Pure Component
Normal JS function	React component
Same input = same output	Same props = same UI
No side effects	Optimized rendering in React

Derived State

a piece of state or value that is calculated from props or other state, instead of storing it direclty.

"Derived state" wo hota hai jo **directly calculate ho sakta hai**, instead of storing it separately in state.

```
import React, { useState } from 'react'

const DerivedState = () => {
    const[user,setUser] = useState("");
    const[users,setUsers] = useState([]);

    const handleClick=(e)=>{
        setUsers([...users,user]);
    }
  return (
    <div>

        <h3>Derived State Explanation</h3>

        <h4>Total User :{users.length}</h4>
        <h4>Last User :{users[users.length -1]}</h4>
        <h4>Unique User :{new Set(users).size}</h4>
        <p>{user}</p>
        <input type="text" placeholder='Enter your name'
        onChange={(e)=>{setUser(e.target.value)}}/>
        <br /><br />
        <button onClick={handleClick}>Submit</button>
        <div>
            {
                users.map((user, index) => (
                    <p key={index}>{user}</p>
                ))
            }
        </div>

    </div>
  )
}

export default DerivedState
```

What is **Lifting State Up** in React?

Lifting state up means:

Moving the state from a **child component** to their **common parent** so that **multiple components can share and update it**.

Simple Example (Real-life Analogy):

Imagine 2 children playing with the same ball. If **each has their own ball**, they won't know what the other is doing.

But if the **ball is with their parent**, and they request the parent to pass the ball, then they both stay in sync.

In React terms:

- When **two child components** need to **communicate or share data**, move the state **up to their common parent**.
- Then pass it **down as props**, and update it using **functions passed as props**.

Updating Object in react

In React, when you're updating an object stored in state, **never mutate it directly**. Instead, always **create a new object** using the spread operator (...) to ensure React knows the state has changed.

{deep copy ani shallow copy ch lecture bagha lagate }

```jsx
import React, { useState } from 'react';

const Object = () => {
  const [data, setData] = useState({
    Name: 'Saharsh',
    address: {
      city: 'Delhi',
      country: 'India',
    },
  });

  const handleName = (name) => {
    setData((data) => ({
        ...data,Name:name
    }));
  };
  const handleCity =(city)=>{
    setData((data)=>({
        ...data,
        address:{
            ...data.address,
            city:city
        }
    }))
  }
  const handleCountry = (country)=>{
    setData((data)=>({
        ...data,
        address:{
            ...data.address,
            country:country
        }
    }))
  }

  return (
    <div>
      <h3>Updating Object in a State</h3>

      <input type="text" placeholder="Enter Name" onChange={(e) => handleName(e.target.value)}/>
      <input type="text" placeholder='Enter city' onChange={(e)=> handleCity(e.target.value)}/>
      <input type="text" placeholder='Enter country name' onChange={(e)=> handleCountry(e.target.value)}/>
      <h2>Name : {data.Name}</h2>
      <h2>City : {data.address.city}</h2>
      <h2>Country : {data.address.country}</h2>
    </div>
  );
};

export default Object;
```

Updating the Array in React

iska bhi same he hai directly change nahi kr sakte

```
import React, { useState } from 'react';

const Array = () => {
  const [users, setUsers] = useState(["saharsh", "mahesh"]);
  const [input, setInput] = useState("");         // New user add karne ke liye
  const [editIndex, setEditIndex] = useState(null); // Kis user ko edit kar rahe hain
  const [editInput, setEditInput] = useState("");   // Edit ke liye input value

  // ✅ Add User
  const handleAddUser = () => {
    if (input.trim()) {
      setUsers(prev => [...prev, input]);
      setInput("");
    }
  };

  // ✅ Edit User
  const handleEdit = (index) => {
    setEditIndex(index);
    setEditInput(users[index]);
  };

  const handleSave = () => {
    setUsers(prevUsers =>
      prevUsers.map((user, index) =>
        index === editIndex ? editInput : user
      )
    );
    setEditIndex(null);
    setEditInput("");
  };

  // ✅ Delete User
  const handleDelete = (index) => {
    setUsers(prev => prev.filter((_, i) => i !== index));
  };

  return (
    <div style={{ padding: "20px", fontFamily: "Arial" }}>
      <h2>User Management</h2>

      {/* Add User Input */}
      <input
        type="text"
        value={input}
        placeholder="Enter user name"
        onChange={(e) => setInput(e.target.value)}
      />
      <button onClick={handleAddUser}>Add User</button>

      <hr />

      {/* User List */}
      {users.map((user, index) => (
        <div key={index} style={{ marginBottom: "10px" }}>
          {editIndex === index ? (
            <>
              <input
                value={editInput}
                onChange={(e) => setEditInput(e.target.value)}
              />
              <button onClick={handleSave}>Save</button>
              <button onClick={() => setEditIndex(null)}>Cancel</button>
            </>
          ) : (
```

```
          <>
            <span>{user}</span>
            <button onClick={() => handleEdit(index)} style={{ marginLeft: "10px" }}>Edit</button>
            <button onClick={() => handleDelete(index)} style={{ marginLeft: "5px" }}>Delete</button>
          </>
        )}
      </div>
    ))}
  </div>
  );
};

export default Array;
```